

Part 1 进程同步概念

1.2 临界资源

■ count++:

- R1=count; 1
- R1=R1+1; 2
- count=R1; 3

■ count--:

- R2=count; 4
- R2=R2-1; 5
- count=R2; 6

count++和count--的机器语言

Count初值为4,并发执行

执行次序	结果	是否正确
123456	4	是
142536	3	否
145263	5	否

解决方法: 下列语句必须被原子性地执行

counter++;

counter--;

Part 1 进程同步概念

1.3 临界区

- **临界区**：进程中涉及临界资源的代码段
- **进入区**：用于检查是否可以进入临界区的代码段。
- **退出区**：将临界区正被访问的标志恢复为未被访问标志。
- **剩余区**：其他代码

一个访问临界资源的循环进程的描述

```
While(TRUE){  
    进入区  
    临界区  
    退出区  
    剩余区  }
```

Part 1 进程同步概念

1.3 临界区

● 例题2

一个正在访问临界资源的进程，由于申请等待I/O操作而被中断时，_____。

- ~~A.~~ 可以允许其他进程进入与该进程相关的临界区。
- ~~B.~~ 不允许其他进程进入任何临界区。
- C. 可以允许其他就绪进程抢占处理器，继续运行。
- ~~D.~~ 不允许任何进程抢占处理器。

Part 1 进程同步概念

1.4 同步机制应遵循的准则

- ❖ 空闲则入：其他进程均不处于临界区,则允许进入临界区；
- ❖ 忙则等待：已有进程处于其临界区则等待；
- ❖ 有限等待：等待进入临界区的进程不能"死等"；
- ❖ 让权等待：不能进入临界区的进程，应释放CPU（如转换到阻塞状态）

Part 2 进程同步机制



目标评价

1. 分析软件、硬件、信号量机制的特点
2. 信号量如何定义？PV操作实现的功能？
3. 如何用信号量实现进程互斥和同步
4. 用信号量机制分析解决实际问题
5. 设计程序用信号量实现线程之间的同步

Part 2 进程同步机制

2.1 软件同步机制

软件同步机制

硬件同步机制

信号量机制

管程机制



- 使用编程方法解决临界区问题
- 有难度、具有局限性，现在很少采用



- 使用特殊的硬件指令，可有效实现进程互斥



- 一种有效的进程同步机制，已被广泛应用



- 新的进程同步机制

Part 2 进程同步机制

2.1 软件同步机制

● Dekker算法

turn: 0 P0进入临界区
 1 P1进入临界区

....

```
P0 : do {  
    while (turn!=0) ;  
    critical section  
    turn:=1;  
    remainder section  
} while(1);
```

```
P1 : do {  
    while (turn!=1) ;  
    critical section  
    turn:=0;  
    remainder section  
}while(1);
```

- ❖ 空闲则入
- ❖ 忙则等待
- ❖ 有限等待
- ❖ 让权等待

Part 2 进程同步机制

2.1 软件同步机制

● Perterson算法

flag:array[0..1] of boolean;

P0: do

```
{ flag[0]:=true;  
while flag[1] ;  
critical section  
flag[0]:=false;  
remainder section  
}while(1);
```

P1: do

```
{ flag[1]:=true;  
while flag[0] ;  
critical section  
flag[1]:=false;  
remainder section  
}while(1);
```


Part 2 进程同步机制

2.1 软件同步机制

● 例题3

进行P0和P1的共享变量定义及其初值为:

```
boolean flag[2];
```

```
int turn=0;
```

```
flag[0]=false; flag[1]=false;
```

Part 2 进程同步机制

2.1 软件同步机制

若进行P0和P1访问临界资源的类C代码如下:

<pre>void p0() // 进程p0 { while (TRUE) { flag[0]=TRUE; turn=1; While (flag[1]&&(turn==1)); 临界区; flag[0]=FALSE; } }</pre>	<pre>void p1() // 进程p1 { while (TRUE) { flag[1]=TRUE; turn=0; While (flag[0]&&(turn==0)); 临界区; flag[1]=FALSE; } }</pre>
---	---

turn=0;
flag[0]=false; flag[1]=false;

则并发执行进程P0和P1时产生的情况是

- (A)不能保证进程互斥进入临界区,会出现“饥饿”现象
- (B)不能保证进程互斥进入临界区,不会出现“饥饿”现象
- (C)能保证进程互斥进入临界区,会出现“饥饿”现象
- (D)能保证进程互斥进入临界区,不会出现“饥饿”现象

Part 2 进程同步机制

2.1 软件同步机制

- 软件解法的缺点

1. 忙等待

2. 实现过于复杂，需要高的编程技巧

Part 2 进程同步机制

2.2 硬件同步机制

1. 关中断

repeat

“屏蔽中断” 指令

<临界区>

“开中断” 指令

until false

Part 2 进程同步机制

2.2 硬件同步机制

2. “测试并设置” 指令

```
boolean TS (boolean *lock)
{
    boolean old;

    old=*lock;

    *lock=TRUE;

    return old;
}
```

➤ 共享数据:

boolean lock = FALSE;

➤ 进程 P_i

```
do {
    while (TS(lock)) ;

    critical section

    lock = FALSE;

    remainder section
}
```

Part 2 进程同步机制

2.2 硬件同步机制

- 3.swap指令

- 原子地交换两个变量

```
void Swap(boolean *a, boolean *b)
{
    boolean temp = *a;
    *a = *b;
    *b = temp;
}
```

- 共享数据 (初始化为 false):

```
boolean lock;
boolean waiting[n];
```

- 进程 P_i

```
do {
    key = true;
    do {
        Swap(lock, key);
    } while (key != false)
    临界区
    lock = false;
    剩余区
} while (true)
```

Part 2 进程同步机制

2.3 信号量机制

迪科斯彻(1930-2002) (Edsger Wybe Dijkstra)



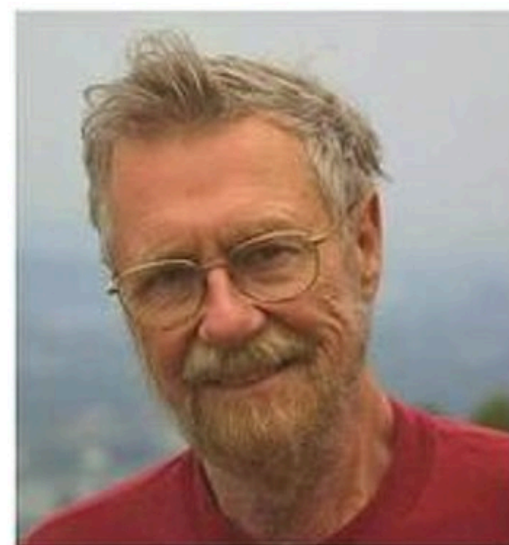
➤ 信号量和PV原语发明者

➤ 解决了“哲学家就餐”问题

➤ 最短路径算法(SPF)和银行家算法的创造者

➤ 结构程序设计之父

➤ THE操作系统设计者和开发者



与D. E. Knuth并称为这个时代最伟大的计算机科学家

Part 2 进程同步机制

2.3 信号量机制

- **信号量 (semaphore)**

1965年，荷兰学者Dijkstra提出（P、V分别代表荷兰语的Proberen (test)和Verhogen (increment)），是一种卓有成效的进程同步机制。

Part 2 进程同步机制

2.3 信号量机制

- 信号量 (semaphore)

整型变量 s

P操作 $s = s - 1$

V操作 $s = s + 1$

P - Test
V - Add

Part 2 进程同步机制

2.3 信号量机制

整型信号量

➤ 信号量S-整型变量

➤ 提供两个不可分割的[原子操作]访问信号量

wait(S):

```
while s <= 0 ; /*do no-op*/  
s := s - 1;
```

signal(S):

```
s := s + 1;
```

➤ Wait(s)又称为P(S)

➤ Signal(s)又称为V(S)

➤ 缺点：进程忙等

P(S)
V(S)

Part 2 进程同步机制

2.3 信号量机制

记录型信号量:去除忙等的信号量

每个信号量S除一个整数值S.value外, 还有一个进程等待队列S.list, 存放阻塞在该信号量的各个进程PCB

- 信号量只能通过初始化和两个标准的原语PV来访问 - - 作为OS核心代码执行, 不受进程调度的打断

```
typedef struct {  
    int value;  
    struct process_control_block *list;  
}semaphore;
```

Part 2 进程同步机制

2.3 信号量机制

记录型信号量: 去除忙等的信号量

- wait操作 (P操作)

wait(s)

{

s.value--;

if (s.value < 0)

{

该进程状态置为等待状态

将该进程的PCB插入相应的等待队列s.queue末尾;

}

}

Part 2 进程同步机制

2.3 信号量机制 记录型信号量:去除忙等的信号量

- signal操作 (V操作)

signal(s)

{

 s.value ++;

 if (s.value <= 0)

 {

 唤醒相应等待队列s.queue中等待的一个进程
 改变其状态为就绪态, 并将其插入就绪队列

 }

}

Part 2 进程同步机制

2.3 信号量机制 信号量的应用

- ① 利用信号量实现进程互斥
 - 设置互斥信号量
- ② 利用信号量实现前趋关系/进程同步
 - 设置同步信号量

Part 2 进程同步机制

2.3 信号量机制 信号量的应用

● 信号量使用要点

- ❖ 必须置一次且**只能置一次**初值
- ❖ 初值**不能为负数**
- ❖ 只能**执行P、V操作**

Part 2 进程同步机制

2.3 信号量机制 信号量的应用

- 利用信号量实现互斥

引例2-11 两个进程互斥访问公用变量count

Cobegin process p1

{

P操作

临界区

V操作

}

Coend.

Cobegin process p2

{

P操作

临界区

V操作

}

Coend.

信号量
初值?

Part 2 进程同步机制

2.3 信号量机制

```
semaphore mutex;
```

```
mutex=1; // 初始化为 1
```

```
while(1)
```

```
{
```

```
    wait(mutex);
```

```
    临界区;
```

```
    signal(mutex);
```

```
    剩余区;
```

```
}
```

Part 2 进程同步机制

2.3 信号量机制

- 信号量实现互斥方法:

- ❖ 为临界资源设置一个互斥信号量mutex(MUTual Exclusion), 其初值为1; 在每个进程中将临界区代码置于P(mutex)和V(mutex)原语之间
- ❖ 必须成对使用P和V原语: 遗漏P原语则不能保证互斥访问, 遗漏V原语则不能在使用临界资源之后将其释放(给其他等待的进程); P、V原语不能次序错误、重复或遗漏

Part 2 进程同步机制

2.3 信号量机制

Q2:如何用信号量实现进程互斥?

- 如何定义信号量?
- 信号量初值如何设置?
- PV操作如何放置?